2026

MODUL PRAKTIKUM 2

AUTOGRAD AND GRADIENT OPTIMIZATION

OPTIMIZED VERSION

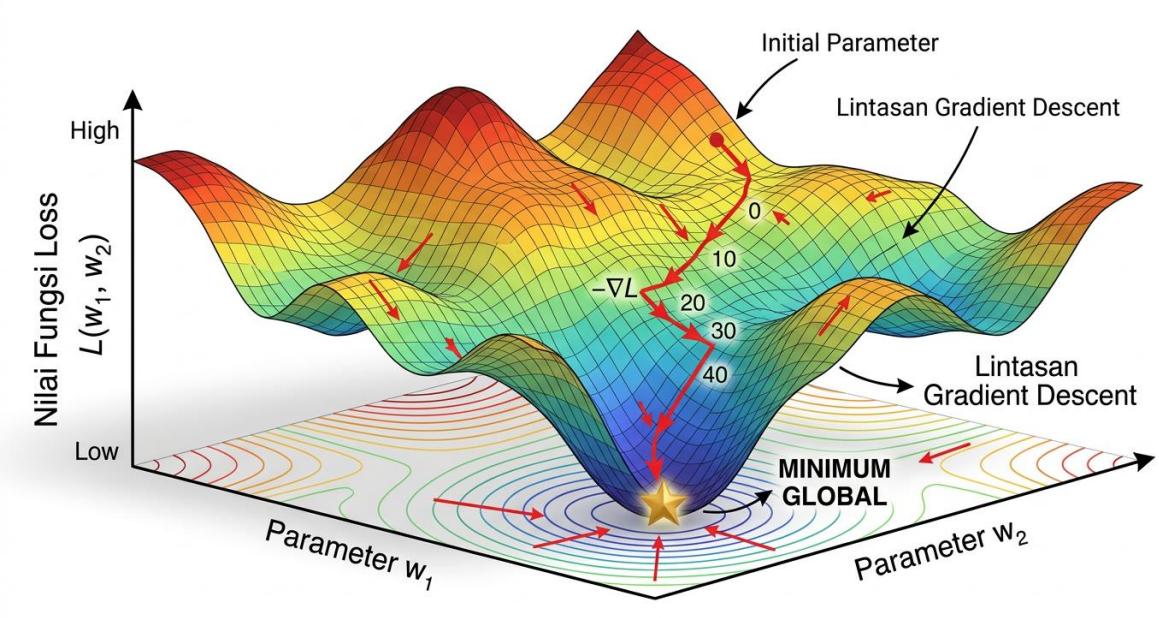
Disusun Oleh:
Ryan F F Hakim

Dipersembahkan Untuk:
Sains Data

Mata Kuliah
Deep Learning

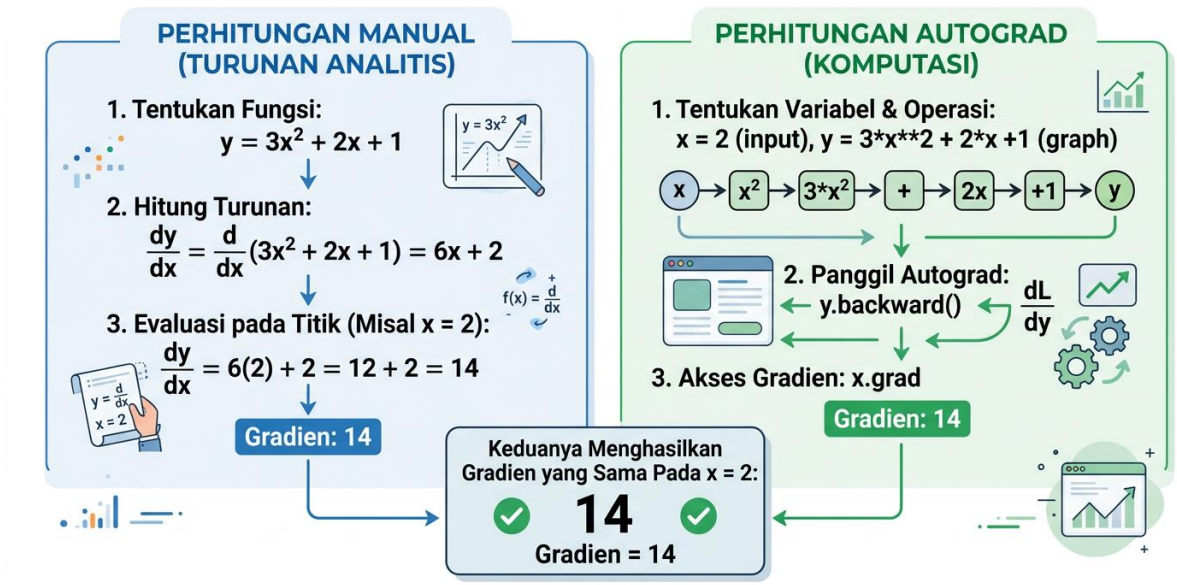
PENDAHULUAN

Dalam pembelajaran mesin mendalam (Deep Learning), tujuan utama pelatihan model adalah menemukan nilai parameter (bobot dan bias) yang dapat meminimalkan fungsi loss. Proses pencarian nilai optimal ini dilakukan melalui teknik optimasi berbasis gradient. Gradient menunjukkan arah dan besarnya perubahan yang harus dilakukan pada setiap parameter agar fungsi loss dapat diturunkan. Oleh karena itu, kemampuan untuk menghitung gradient secara cepat, akurat, dan efisien menjadi salah satu pilar utama dalam pengembangan model deep learning.



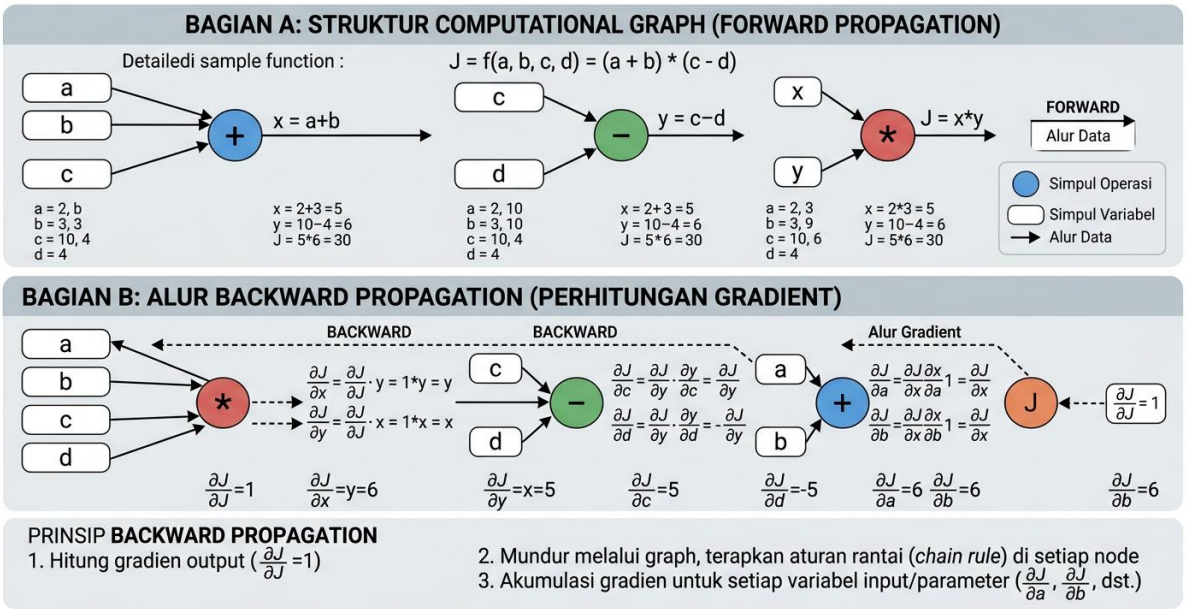
Gambar 1.1: Ilustrasi permukaan fungsi loss tiga dimensi beserta arah gradient descent

Secara tradisional, menghitung turunan fungsi loss terhadap ribuan hingga jutaan parameter secara manual sangatlah rumit dan rawan kesalahan. Untuk mengatasi masalah ini, dikembangkanlah sistem **Autograd (Automatic Differentiation)**. Autograd adalah teknik komputasi yang secara otomatis membangun *computational graph* selama proses *forward pass* dan kemudian menghitung gradient yang dibutuhkan melalui proses *backward pass*. Dengan adanya autograd, peneliti dan praktisi tidak perlu lagi menuliskan rumus derivatif secara eksplisit untuk setiap operasi matematika yang kompleks dalam arsitektur neural network.



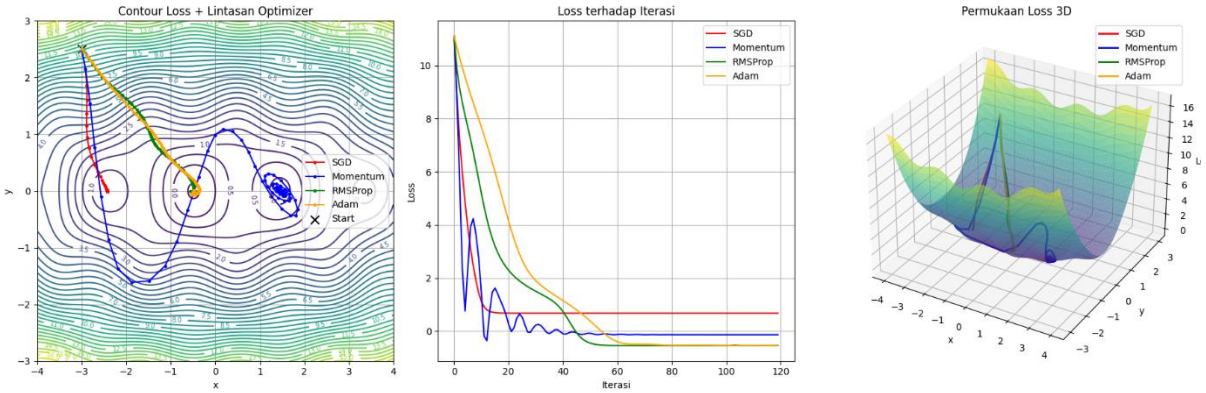
Gambar 1.2: Perbandingan perhitungan gradient secara manual dengan menggunakan autograd

Autograd bekerja berdasarkan prinsip *reverse-mode automatic differentiation*. Setiap operasi tensor dicatat dalam graph komputasi beserta turunannya. Ketika fungsi loss telah dihitung, gradient dapat disebarkan secara efisien dari output menuju semua parameter menggunakan aturan rantai (*chain rule*). Framework seperti PyTorch mengimplementasikan autograd secara native melalui modul torch.autograd, sehingga setiap tensor yang membutuhkan gradient dapat dilacak secara otomatis dengan hanya mengatur requires_grad=True.



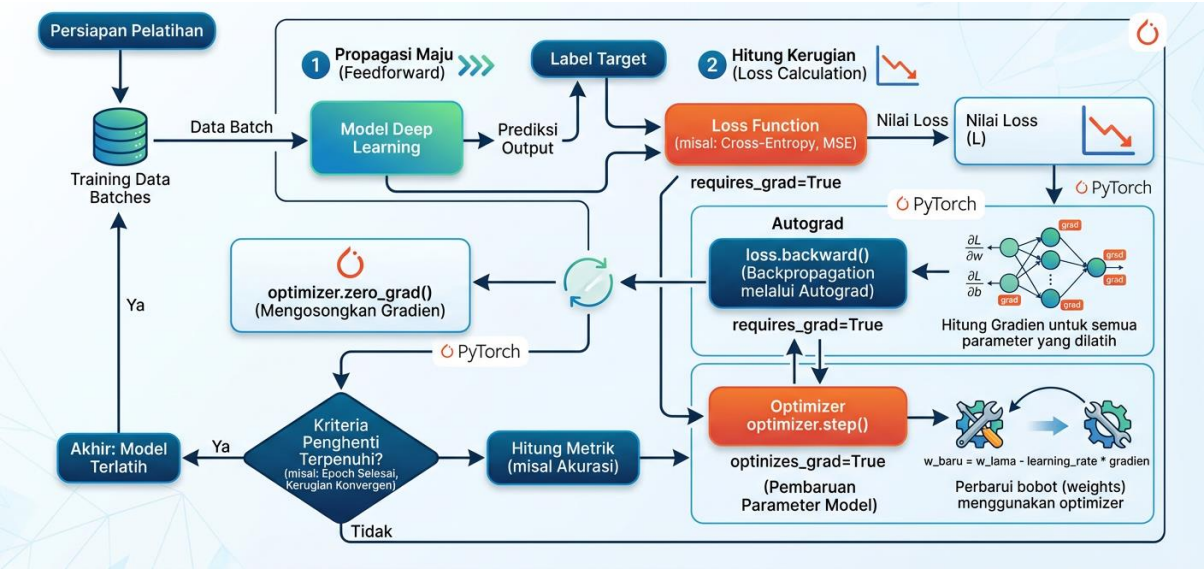
Gambar 1.3: Struktur computational graph dan alur backward propagation

Sementara autograd bertugas menghitung gradient, optimasi gradient bertugas menggunakan informasi gradient tersebut untuk memperbarui parameter model. Berbagai algoritma optimasi telah dikembangkan, mulai dari yang paling sederhana yaitu Gradient Descent, hingga varian yang lebih canggih seperti Stochastic Gradient Descent (SGD), SGD with Momentum, RMSprop, dan Adam. Setiap algoritma memiliki strategi berbeda dalam memanfaatkan gradient untuk menyeimbangkan antara kecepatan konvergensi, stabilitas, dan kemampuan keluar dari local minima.



Gambar 1.4: Perbandingan lintasan optimasi berbagai algoritma pada permukaan loss

Pada modul praktikum Deep Learning ini, Anda akan mempelajari secara mendalam bagaimana autograd bekerja di balik layar serta bagaimana mengimplementasikan dan membandingkan berbagai algoritma optimasi gradient. Pemahaman yang kuat mengenai kedua konsep ini menjadi fondasi penting sebelum melanjutkan ke topik-topik yang lebih advanced seperti arsitektur model, regularisasi, dan teknik pelatihan yang lebih kompleks.



Gambar 1.5: Diagram alur pelatihan model deep learning menggunakan Autograd dan Optimizer di PyTorch

DEPENDENSI

CODE

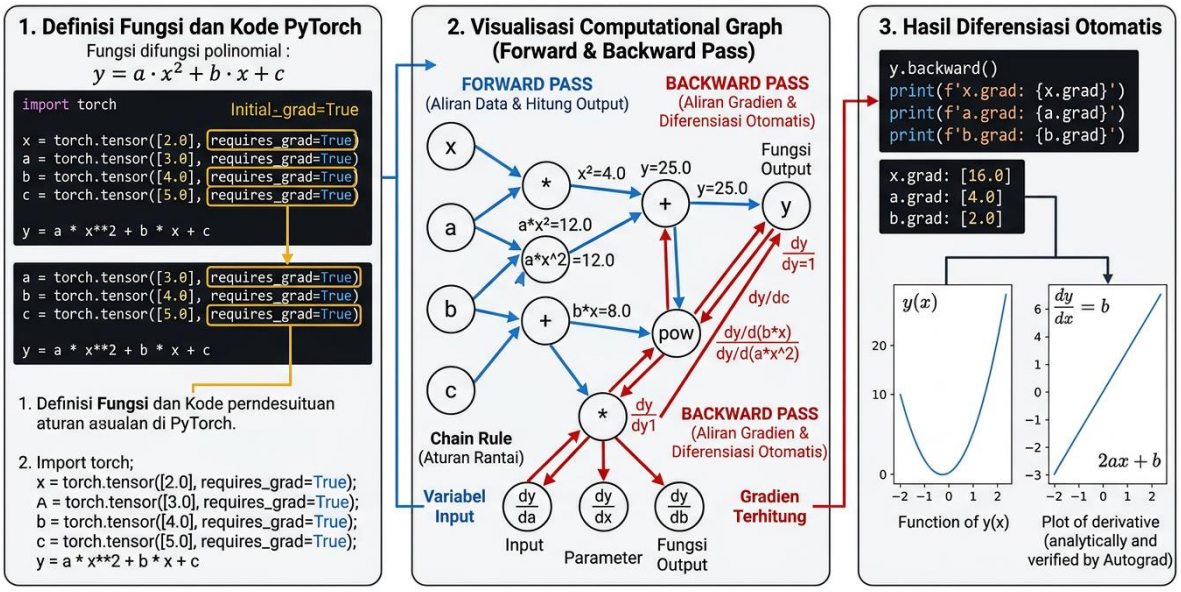
```
!pip install torchviz
```

CODE

```
import torch
import matplotlib.pyplot as plt
import torchviz
import torch.optim as optim
from torchviz import make_dot
from IPython.display import display
```

AUTOGRAD DASAR

1. GRADIENT OTOMATIS (AUTOGRAD)

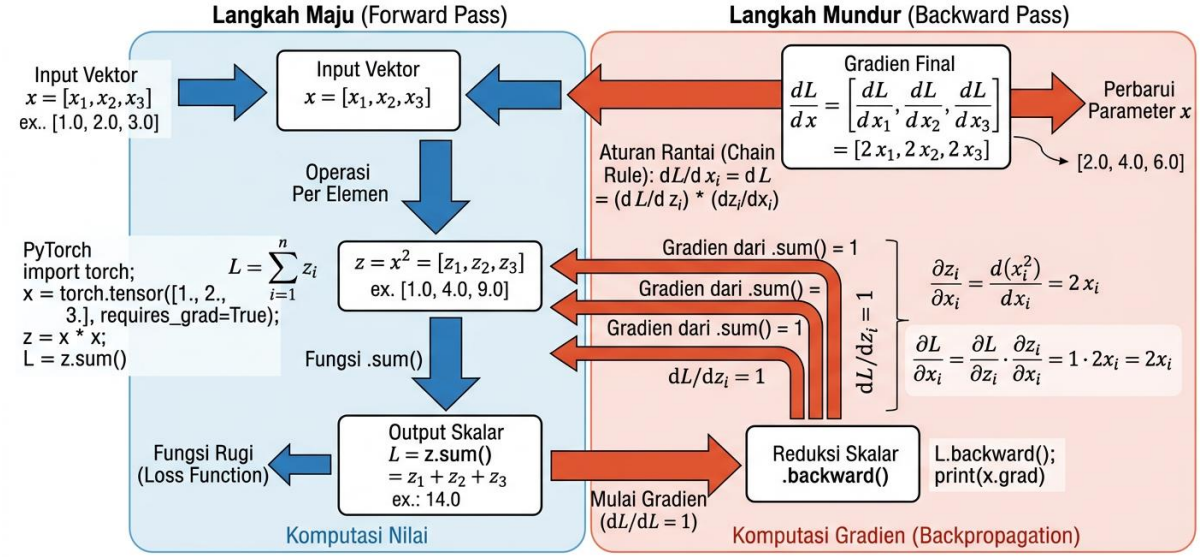


Gambar 2.1: Implementasi computational graph dan diferensiasi otomatis menggunakan pytorch autograd pada fungsi polinomial.

CODE

```
# 1. Gradient Otomatis (Autograd)

# requires_grad=True → PyTorch melacak operasi untuk menghitung gradien
x = torch.tensor(3.0, requires_grad=True)

# Forward pass: definisikan fungsi
# f(x) = x^3 + 2x^2 - 5x + 7
y = x**3 + 2*x**2 - 5*x + 7

print(f"x      = {x}")
print(f"f(x)   = {y}")
print(f"y memiliki grad_fn: {y.grad_fn}")

# Backward pass: hitung gradien
# f'(x) = 3x^2 + 4x - 5
y.backward()

print(f"\nGradien df/dx di x=3:")
print(f"Autograd  : {x.grad}")
print(f"Manual    : {3*3**2 + 4*3 - 5}")  # 3(9) + 12 - 5 = 34
```

2. GRADIENT PADA VEKTOR

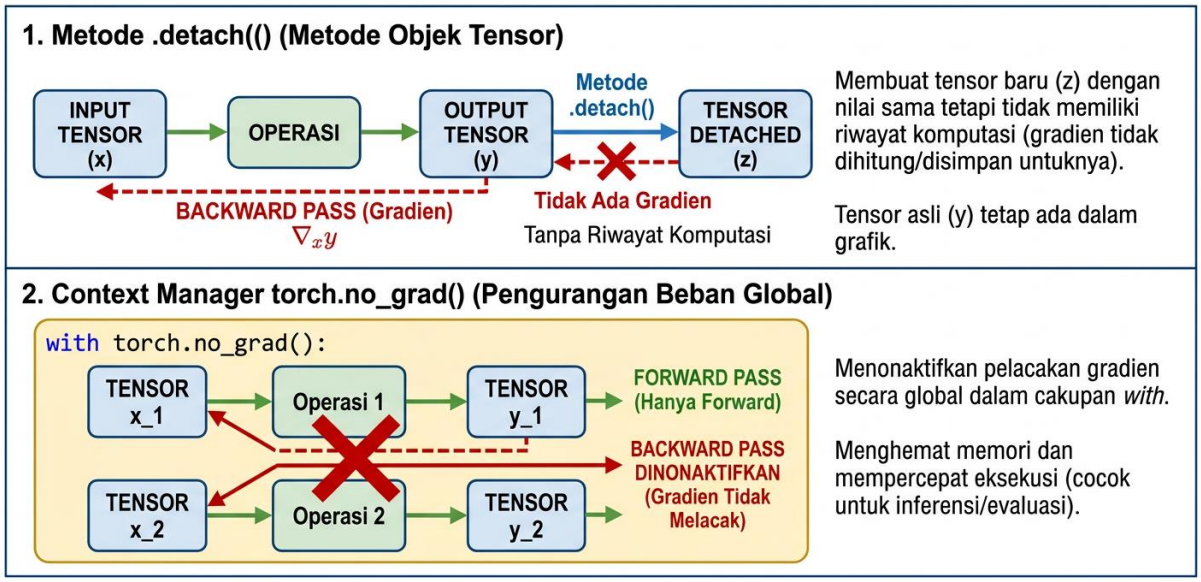


Gambar 2.2: Proses diferensiasi otomatis pada operasi vektor dan reduksi skalar menggunakan fungsi `.sum()`.

CODE

```
# 2. Gradien pada Vektor

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2 + 2 * x
z = y.sum()  # Scalar output diperlukan untuk backward()

z.backward()
print(f"\nx        = {x}")
print(f"y        = {y}")
print(f"dz/dx    = {x.grad}")  # 2x + 2 → [4, 6, 8]
```

3. DETACH DAN NO GRAD

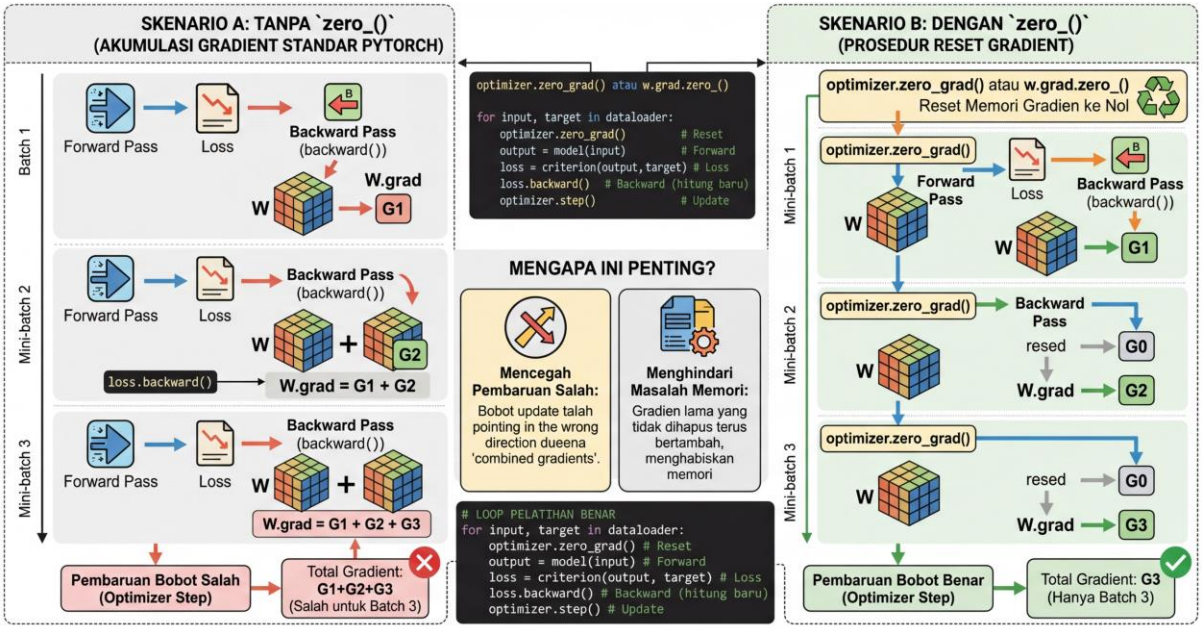


Gambar 2.3: Mekanisme pemutusan hubungan komputasi menggunakan method `.detach()` dan context manager `torch.no_grad()`.

CODE

```
# 3. Detach dan No Grad

x = torch.tensor(2.0, requires_grad=True)
y = x * 3
# Detach: memutus dari computational graph
y_detached = y.detach()
print(f"\ny (requires_grad) : {y.requires_grad}")
print(f"y_detached         : {y_detached.requires_grad}")

# torch.no_grad(): context manager untuk inferensi
with torch.no_grad():
    z = x * 5
    print(f"z (no_grad)        : {z.requires_grad}")
```

4. AKUMULASI GRADIENT DAN RESET

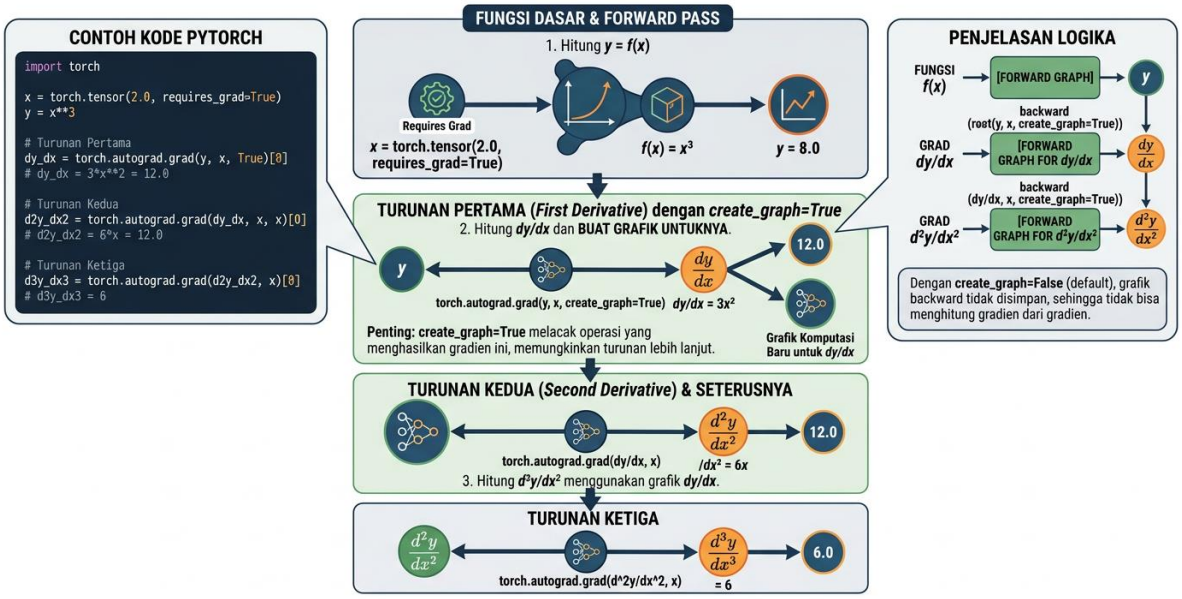


Gambar 2.4: Mekanisme akumulasi gradien pada methode `.zero_grad()` dan pentingnya prosedur untuk reset memori gradien

CODE

```
# 4. Akumulasi Gradien dan Reset

x = torch.tensor(2.0, requires_grad=True)

# Iterasi 1
y1 = x ** 2
y1.backward()
print(f"\nGrad setelah iter 1 : {x.grad}")  # 4

# Iterasi 2 (TANPA reset → akumulasi!)
y2 = x ** 3
y2.backward()
print(f"Grad setelah iter 2 : {x.grad}")  # 4 + 12 = 16 (SALAH!)

# PENTING: Reset gradien sebelum backward baru
x.grad.zero_()
y3 = x ** 3
y3.backward()
print(f"Grad setelah reset  : {x.grad}")  # 12 (BENAR)
```

5. TURUNAN ORDE TINGGI

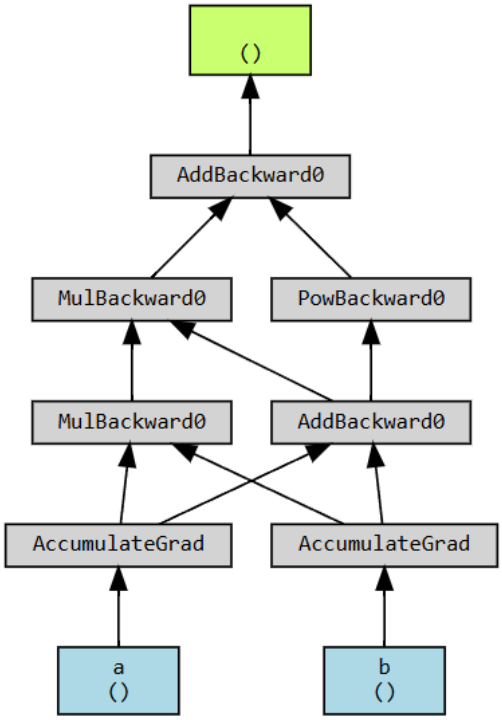


Gambar 2.4: Implementasi komputasi turunan orde tinggi pada fungsi $f(x) = x^3$ menggunakan fitur `create_graph=True` di PyTorch untuk melacak gradien dari gradien sebelumnya.

CODE

```
# 5. Turunan orde tinggi (create_graph=True)

x = torch.tensor(2.0, requires_grad=True)

# f(x) = x³
y = x ** 3
print(f"f(x) = x³, f(2) = {y.item()}")

# Turunan pertama: f'(x) = 3x²
grad1 = torch.autograd.grad(y, x, create_graph=True)[0]
print(f"f'(x) = 3x², f'(2) = {grad1.item()}")  # 12

# Turunan kedua: f''(x) = 6x
grad2 = torch.autograd.grad(grad1, x, create_graph=True)[0]
print(f"f''(x) = 6x, f''(2) = {grad2.item()}")  # 12

# Turunan ketiga: f'''(x) = 6
grad3 = torch.autograd.grad(grad2, x)[0]
print(f"f'''(x) = 6, f'''(2) = {grad3.item()}")  # 6
```

VISUALISASI COMPUTATIONAL GRAPH

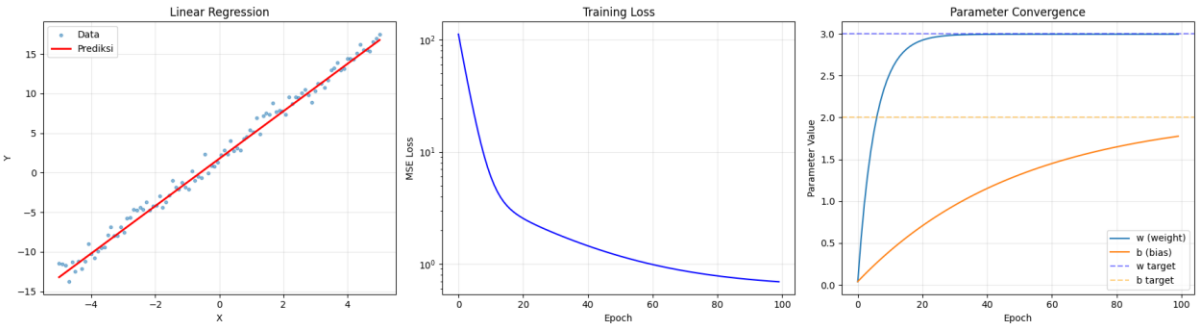


Gambar 3.1: Visulisasi Computational Graph pada PyTorch

CODE

```
# 6. Visualisasi Computational Graph

# Definisikan Operasi
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)

c = a * b            # c = 6
d = a + b            # d = 5
e = c * d + d**2     # e = 6*5 + 25 = 55

# Visualisasi
graph = make_dot(e, params={"a": a, "b": b})
graph.attr(rankdir='BT')
display(graph)

# Hitung gradien
e.backward()
print(f"\nde/da = {a.grad}")  # d(c*d + d^2)/da = b*d + c + 2d = 3*5 + 6
+ 2*5 = 31
print(f"de/db = {b.grad}")  # d(c*d + d^2)/db = a*d + c + 2d = 2*5 + 6 +
2*5 = 26
```

IMPLEMENTASI GRADIENT DESCENT MANUAL



Gambar 4.1: Proses optimasi Linear Regression menggunakan metode Gradient Descent. Panel kiri menunjukkan kesesuaian garis prediksi terhadap data asli; panel tengah menunjukkan penurunan Mean Squared Error (MSE) secara logaritmik; dan panel kanan menunjukkan konvergensi parameter w (weight) dan b (bias) menuju nilai target.

CODE

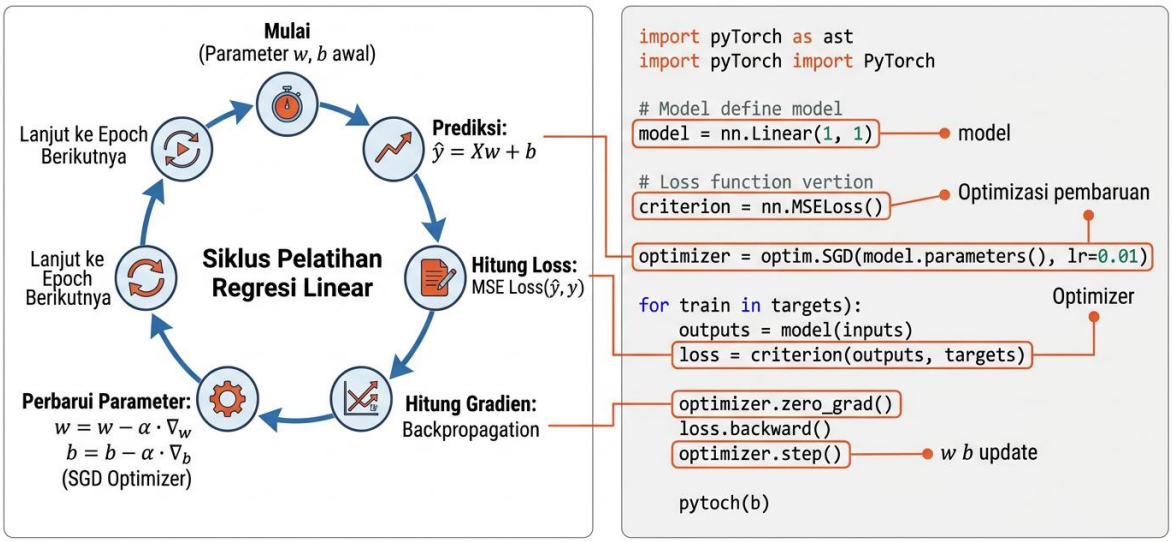
```
# 7. Linear Regression dari nol

torch.manual_seed(42)

# Generate data sintetis: y = 3x + 2 + noise
X_data = torch.linspace(-5, 5, 100).unsqueeze(1)
y_true = 3 * X_data + 2 + torch.randn_like(X_data) * 0.8

# Parameter yang akan dioptimasi
w = torch.randn(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

# Hyperparameters
learning_rate = 0.01
epochs = 100

# Logging
loss_history = []
w_history = []
b_history = []

print("=== Gradient Descent Manual ===")
for epoch in range(epochs):
    # 1. Forward pass
    y_pred = X_data * w + b

    # 2. Hitung loss (MSE)
    loss = ((y_pred - y_true) ** 2).mean()

    # 3. Backward pass
    loss.backward()

    # 4. Update parameter (TANPA tracking gradient)
    with torch.no_grad():
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad
```

```
    # 5. Reset gradient
    w.grad.zero_()
    b.grad.zero_()

    # Logging
    loss_history.append(loss.item())
    w_history.append(w.item())
    b_history.append(b.item())

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1:3d} | Loss: {loss.item():.4f} | "
              f"w: {w.item():.4f} | b: {b.item():.4f}")

print(f"\nHasil akhir: w = {w.item():.4f} (target: 3.0), "
      f"b = {b.item():.4f} (target: 2.0)")
```

CODE

```
# Visualisasi
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Plot 1: Data dan garis regresi
axes[0].scatter(X_data.numpy(), y_true.numpy(), alpha=0.5, s=10,
label='Data')
with torch.no_grad():
    y_final = X_data * w + b
axes[0].plot(X_data.numpy(), y_final.numpy(), 'r-', linewidth=2,
label='Prediksi')
axes[0].set_xlabel('X')
axes[0].set_ylabel('Y')
axes[0].set_title('Linear Regression')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: Loss curve
axes[1].plot(loss_history, 'b-', linewidth=1.5)
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('MSE Loss')
axes[1].set_title('Training Loss')
axes[1].set_yscale('log')
axes[1].grid(True, alpha=0.3)

# Plot 3: Parameter trajectory
axes[2].plot(w_history, label='w (weight)', linewidth=1.5)
axes[2].plot(b_history, label='b (bias)', linewidth=1.5)
axes[2].axhline(y=3.0, color='blue', linestyle='--', alpha=0.5, label='w
target')
axes[2].axhline(y=2.0, color='orange', linestyle='--', alpha=0.5,
label='b target')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Parameter Value')
axes[2].set_title('Parameter Convergence')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('linear_regression_manual.png', dpi=150, bbox_inches='tight')
plt.show()
```

MENGGUNAKAN PYTORCH OPTIMIZER



Gambar 5.1: Implementasi Regresi Linear menggunakan modul torch.optim.

Optimizer dalam deep learning adalah algoritma atau metode yang digunakan untuk memperbarui parameter-parameter model, seperti bobot (*weights*) dan bias, dengan tujuan meminimalkan fungsi kerugian (*loss function*). Dalam proses pelatihan jaringan saraf tiruan, model perlu menyesuaikan jutaan bahkan miliaran parameter agar dapat menghasilkan prediksi yang akurat. Optimizer berperan sebagai mekanisme yang menentukan bagaimana dan seberapa besar parameter-parameter tersebut harus diubah pada setiap iterasi pelatihan berdasarkan nilai gradien yang dihitung melalui proses *backpropagation*.

Berikut adalah beberapa contoh Optimizer yang ada di PyTorch

1. SGD (Stochastic Gradient Descent)
   - Parameter utama: learning rate (η)
   - Persamaan pembaruan:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

dimana θ_t adalah vektor bobot pada iterasi t dan $L(\theta_t)$ adalah fungsi loss.

2. RMSProp
   - Parameter utama: learning rate (η), decay rate (ρ), epsilon (ϵ)
   - Persamaan pembaruan:
     a. Perbarui eksponensial moving average dari kuadrat gradien:

$$v_t = \rho\, v_{t-1} + (1-\rho)\left(\nabla_\theta L(\theta_t)\right)^2$$

     b. Pembaruan bobot:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}}\, \nabla_\theta L(\theta_t)$$

3. Adam (Adaptive Moment Estimation)
   - Parameter utama: learning rate (η), decay rates (β_1), (β_2), epsilon (ϵ)
   - Persamaan pembaruan:
     a. Momentum pertama (rata-rata gradien):

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, \nabla_\theta L(\theta_t)$$

     b. Momentum kedua (rata-rata kuadrat gradien):

$$v_t = \beta_2\, v_{t-1} + (1-\beta_2)\left(\nabla_\theta L(\theta_t)\right)^2$$

     c. Koreksi bias:

$$\widehat{m_t} = \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v_t} = \frac{v_t}{1-\beta_2^t}$$

     d. Pembaruan bobot

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\widehat{v_t}} + \epsilon}\, \widehat{m_t}$$

CODE

```python
# 8. Regresi Linear dengan torch.optim

import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)
X = torch.rand(100, 1) * 10
y_true = 3 * X + 2 + torch.randn(100, 1) * 0.5

# Parameter sebagai nn.Parameter
w = nn.Parameter(torch.randn(1))
b = nn.Parameter(torch.randn(1))

# Optimizer
optimizer = optim.SGD([w, b], lr=0.01)

# Loss function
loss_fn = nn.MSELoss()

# Training loop
for epoch in range(100):
    # Forward
    y_pred = w * X + b
    loss = loss_fn(y_pred, y_true)

    # Backward
    optimizer.zero_grad()    # Reset gradien
    loss.backward()          # Hitung gradien
    optimizer.step()         # Update parameter

    if (epoch + 1) % 20 == 0:
        print(f"Epoch [{epoch+1:3d}/100] | Loss: {loss.item():.4f}")

print(f"\nw = {w.item():.4f}, b = {b.item():.4f}")
```